NOT FOR STUDENTS

Answer Key — Polymorphism

Instructor reference for the review worksheet — page 1 of 3

part 1 The Problem We Left Off With — Q1, Q2

CALL	AS WRITTEN	WITH `virtual`
`(a) c.describe();`	A car.	A car.
`(b) vp→describe();`	A vehicle.	A car.
`(c) vr.describe();`	A vehicle.	A car.

(a) never changes. It's called directly on a `Car` object, not through a pointer or reference, so there is nothing to look up. The compiler already knows the object is a `Car`.

1. **declared · compile · static (early)**
2. **actual · run · dynamic (late)**
3. **pointer · reference**
4. **forms**

part 2 Two Kinds of Polymorphism — Q3, Q4

1. `print(5)` — **COMPILE TIME** (function overloading; chosen from the argument type)
2. `a + b` — **COMPILE TIME** (operator overloading)
3. `maxOf(...)` — **COMPILE TIME** (the template makes an `int` and a `double` version while compiling)
4. `s→draw()` — **RUN TIME** (virtual + base pointer)
5. `v→describe()` non-virtual — **COMPILE TIME**. This is the trap: it's bound to the pointer type, so it runs `Vehicle::describe()` even though the object is a `Car`.
6. `c→attack()` — **RUN TIME**

Q4: The type depends on `rand()`, which only has a value **while the program runs**. The compiler can't know which class it will be, so the language has to make the choice at run time. It does this by looking up the function through the actual object (dynamic binding / the vtable), which is exactly what `virtual` asks for.

part 3 `virtual`, `override` — Q5, Q6

LINE	VERDICT	REASON
(1)	OK	Exact match of a virtual base function
(2)	ERROR	Missing `const`. The base is `printName() const`, so the signatures differ.
(3)	ERROR	Parameter is `double`, base takes `int`. Different signature.
(4)	ERROR	`rest()` isn't virtual in the base, and only virtual functions can be overridden
(5)	ERROR	No virtual `Attack` in the base. C++ is case-sensitive.
(6)	OK	Exact match, so it still overrides. `virtual` is inherited, and `override` is optional (but recommended).

Q6: It runs `Character::printName()`. Without `const`, `Wizard::printName()` is a *new* function that hides the base one, so it never goes into the `printName` slot of the vtable. No error, wrong output. That silent bug is exactly what `override` prevents. (clang does warn: `-Woverloaded-virtual`.)

NOT FOR STUDENTS

Answer Key — Polymorphism

Instructor reference for the review worksheet — page 2 of 3

part 4 Pointers, References, and Slicing — Q7, Q8

(a) The magnificent Merlin

(b) The powerful Conan

(c) Merlin

(d) Conan

(e) The powerful Conan

Q8: Both copy the object by value into a plain `Character` (a by-value parameter, and a `vector<Character>`). That's slicing: the new object really *is* just a `Character`, so virtual has nothing to find. Fix: `void byVal(Character& c)`. (Not `const&` here, because `printName()` isn't a `const` method.)

part 5 One Loop, Many Types — Q9, Q10, Q11

OBJECT	PRINTNAME()	ATTACK()
`Warrior("Conan")`	The powerful Conan	Conan cleaves with an axe!
`Wizard("Merlin")`	The magnificent Merlin	Merlin swings wildly.
`Rogue("Vex")`	Vex	Vex stabs from the shadows!
`Character("Bob")`	Bob	Bob swings wildly.

The Wizard/Rogue rows are the point: a class that doesn't override a function falls back to the inherited `Character` version.

Q10 — polymorphic: write one new class, `Bard`, plus one line in the factory (`makeCharacter`) if you use one. Nothing else changes. **if/else:** add a `"bard"` branch to `attack()`, check `printName()`, and then do the same in *every* other function that tests `type` (`defend`, `levelUp`, `save`, …).

Does the loop change? **No.** Forget a branch in the if/else version and there's **no compiler error**: the Bard silently does nothing, or gets the wrong behavior. With a pure virtual (Part 7), a forgotten method becomes a compile error.

Q11 model answer:

```
class Bard : public Character {
public:
    Bard(string n) : Character(n) {}
    void attack() override { cout << name << " plays a very loud lute!" << endl; }
};

party.push_back(make_unique<Bard>("Jaskier"));
```

Accept `"\n"` for `endl`. Reaching `name` works because it's `protected` in `Character` (W05). Common mistakes: forgetting `public` in the inheritance list (private inheritance, so the `unique_ptr<Character>` conversion fails), or trying to assign `name` in the constructor body instead of chaining.

part 6 Virtual Destructors — Q12, Q13, Q14

1. ~Base
2. the `new int[100]` array owned by `Derived`
3. `virtual ~Base()`, in class `Base`
4. ~Derived ~Base

Q13: No problem. `d` is a `Derived` object, not something reached through a `Base*`, so the compiler knows its exact type and runs `~Derived` then `~Base`. The bug only appears when you `delete` through a base pointer.

Q14: "If a class has any virtual function, give it a virtual destructor."

NOT FOR STUDENTS

Answer Key — Polymorphism

Instructor reference for the review worksheet — page 3 of 3

part 7 Abstract Classes — Q15–Q18

1. `Shape s("thing");` — **N** (can't create an object of an abstract class)
2. `Shape* p = new Circle(2.0);` — **Y** (pointer to abstract is fine; the object is a concrete `Circle`)
3. `Shape& ref = c;` — **Y**
4. `Blob b;` — **N**
5. `c.name()` — **Y** (prints `circle`; ordinary inherited method)
6. `vector<Shape*> shapes;` — **Y**

Blob: it never overrides the pure virtual `area()`, so `Blob` is **abstract too**. Adding `double area() const override { ... }` makes it concrete. **Q16: pure virtual · interface**

Q17 model answer:

```
class Rectangle : public Shape {
    double width;
    double height;
public:
    Rectangle(double w, double h)
        : Shape("rectangle"),
          width(w), height(h) {}

    double area() const override {
        return width * height;
    }
};
```

Q18 model answer:

```
double totalArea(const vector<Shape*>& shapes) {
    double sum = 0;
    for (Shape* s : shapes)
        sum += s->area();
    return sum;
}
```

Returns **9.14159** (3.14159 + 6).

Q17: watch for a missing `const` on `area()` (see Case B). Q18: the function should never mention `Circle` or `Rectangle`.

part 8 Find the Error — Cases A–D

Case A Prints **"..."**. `speak()` is **not** virtual in `Animal`, and `a` is an `Animal&`, so the call is bound at compile time to `Animal::speak()`. `virtual` has to be on the **base** class's function. Adding it in `Dog` only affects classes derived *from* `Dog`. Fix: `virtual void speak()` in `Animal` (and `override` in `Dog`).

Case B Does **NOT compile**: "variable type 'Square' is an abstract class." `Square::area()` is missing `const`, so it doesn't override `area() const`. The pure virtual is still unimplemented, which leaves `Square` abstract. `override` on the `area()` line would have reported the real problem right where it happened. Fix: `double area() const override`.

Case C **Memory leak.** Every `new` needs a `delete`, and none happen. Fix 1: `for (auto c : party) delete c;` before `main` ends (this relies on `Character` having a virtual destructor, Part 6). Fix 2 (preferred): `vector<unique_ptr<Character>>` with `make_unique<Wizard>("Merlin")`, which cleans up automatically.

Case D Does **NOT compile**: "no member named 'castSpell' in 'Character'." The **pointer's** type decides what you may call, and `Character` has no `castSpell`. `c->attack()` is fine. Fix: keep a `Wizard*` when you need Wizard-only behavior, or add a virtual `specialMove()` to `Character`.

part 9 Under the Hood — Q19, Q20

VTABLE	PRINTNAME	ATTACK
`Character`	Character::	Character::
`Warrior`	Warrior::	Warrior::
`Wizard`	Wizard::	Character::
`Rogue`	Character::	Rogue::

1. **F**: one vtable per *class*, shared. Each object carries only a vptr.
2. **T**: the hidden vptr gets added (typically 8 bytes).
3. **F**: non-virtual calls are resolved at compile time.
4. **F**: the constructor of the *actual* object sets the vptr.
5. **T**: load the vptr, load the slot, then call. It's tiny, but it's real.